7. Write a program to implement Logistic Regression (LR) algorithm in python

Code:

```
datasets = [
    ("Student Placement", placement_X, placement_y, placement_new),
    ("Loan Approval", loan_X, loan_y, loan_new),
    ("Disease Diagnosis", disease_X, disease_y, disease_new),
    ("Employee Promotion", employee_X, employee_y, employee_new),
…     model = make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=1000)
    )

    # Train
    model.fit(X, y_encoded)
# Training prediction
    y_pred = model.predict(X)
…     print("==========================================")
    print("Accuracy:", round(accuracy * 100, 2), "%")
    print("New Sample Prediction:", result)
```

output:

```
==========================================
Student Placement
==========================================
Accuracy: 100.0 %
New Sample Prediction: 0

==========================================
Loan Approval
==========================================
Accuracy: 100.0 %
New Sample Prediction: 1

==========================================
Disease Diagnosis
==========================================
Accuracy: 100.0 %
New Sample Prediction: 1

==========================================
Employee Promotion
==========================================
Accuracy: 100.0 %
New Sample Prediction: 0

==========================================
Fruit Classification
==========================================
Accuracy: 100.0 %
New Sample Prediction: orange
```

8. Write a program to implement Linear Regression (LR) algorithm in python

Code:

LINEAR REGRESSION

```
# Predict Performance Score
# ================================================


# X = Experience and Training Hours
X = employee_X[:, [0, 2]]
# y = Performance Score
y = employee_X[:, 1]


# Create Linear Regression model
model = LinearRegression()


# Train model
model.fit(X, y)


…      round(model.intercept_, 2))
```

Output

```
========================================
LINEAR REGRESSION
========================================
Predicted Performance Score: 198.0
R2 Score: 1.0
Coefficient:
[2. 4.]
Intercept: 8.0
```

9. Compare Linear and Polynomial Regression using Python

Code:

```
# Input variables
X = employee_X[:, [0, 2]]

# Target
y = employee_X[:, 1]

# New sample
new_sample = np.array([[7, 44]])

# ------------------------------
…# ------------------------------
# Polynomial Regression
# ------------------------------

poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
poly_model.fit(X, y)
…print("\nPolynomial Regression")
print("------------------------------")
print("Predicted Performance Score:",
      round(poly_prediction, 2))
print("R2 Score:",
      round(poly_r2, 4))
print("\nComparison")
print("------------------------------")

if poly_r2 > linear_r2:
    print("Polynomial Regression gives better R2 score.")
```

else:

print("Linear Regression gives better R2 score.")

output:

```
==========================================
LINEAR VS POLYNOMIAL REGRESSION
==========================================

Linear Regression
----------------------------
Predicted Performance Score: 198.0
R2 Score: 1.0

Polynomial Regression
----------------------------
Predicted Performance Score: 659.55
R2 Score: 1.0

Comparison
----------------------------
Linear Regression gives better R2 score.
```

10. Write a Python Program to Implement Expectation & Maximization Algorithm

Program:

```
# Use fruit dataset

X = fruit_X

# Standardize data

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


# Gaussian Mixture Model

# EM algorithm is used internally

em_model = GaussianMixture(

   n_components=2,

   random_state=42

…print("==========================================")


print("\nCluster assignments:")


print(clusters)


print("\nCluster means:")

print(em_model.means_)


print("\nNumber of iterations:")

print(em_model.n_iter_)
```

output:

```
==========================================
EXPECTATION MAXIMIZATION ALGORITHM
==========================================

Cluster assignments:
[0 0 0 1 1]

Cluster means:
[[-0.70706938 -0.70706938]
 [ 1.06067393  1.06067393]]

Number of iterations:
2
```